

Exercise 3 Brightlearn

Question 1- The products Table

▶ Run selected (1000) ✓ Just now (1s) workspace default

⋮ Edit

🟢

📅

📄

💬

📤

```
--QUESTION1:
SELECT product_name,
       price,
       CASE
         WHEN price > 1000 THEN 'Expensive'
         WHEN price BETWEEN 100 AND 1000 THEN 'Mid-range'
         WHEN price < 100 THEN 'Budget'
       END AS price_category
FROM exercise3.product table.product;
```

Add parameter

Table +

🔍 🔍 📄 📄 ⌵ ✕

	product_name	price	price_category
1	Laptop	1200	Expensive
2	Phone	800	Mid-range
3	Keyboard	45	Budget
4	Monitor	300	Mid-range
5	Mouse	25	Budget

📄

5 rows

0.537s runtime

[See performance \(1\)](#)

✓ Checked for improvements

Refreshed Now

Question –2 The orders Table

```
(5, 'Ethan', 1200.00);

SELECT*
FROM exercise3.orders_table.orders;

-- Question 2:
SELECT customer_name,
       amount,
CASE
  ... WHEN amount >= 1000 THEN 'High value'
  ... WHEN amount BETWEEN 500 AND 999.99 THEN 'Medium value'
  ... WHEN amount < 500 THEN 'Low value'
END AS order_value_category
FROM exercise3.orders_table.orders;

--3 THE EMPLOYEES TABLE
CREATE SCHEMA IF NOT EXISTS exercise3.employees_table;
```

Add parameter

Table ▼ + 🔍 🏠 📄 🔖

	customer_name	amount	order_value_category
1	Alice	150	Low value
2	Bob	560	Medium value
3	Charlie	1000	High value
4	Diana	46	Low value
5	Ethan	1200	High value

⬇ ▼ 5 rows | 0.471s runtime | [See performance \(1\)](#) | ✓ Checked for improvements

Question 3- The employees Table

```
SELECT*
FROM exercise3.employees_table2.employees;

--Question3: categories each employee
SELECT emp_name,
       department,
       salary,
       CASE
         WHEN department='IT' AND salary>80000 THEN 'Senior IT'
         WHEN department='HR' AND salary>55000 THEN 'Experienced HR'
         ELSE 'Staff'
       END AS position_level
FROM exercise3.employees_table2.employees;
```

Add parameter

Table + 🔍 ⚙️ 📄 🗑️ ^

	emp_name	department	salary	position_level
1	John	IT	85000	Senior IT
2	Sarah	HR	60000	Experienced HR
3	Mark	IT	75000	Staff
4	Lucy	Finance	95000	Staff
5	Tom	HR	55000	Staff

⬇️ ▾

5 rows | 0.677s runtime | [See performance \(1\)](#) | ✓ Checked for improvements

Refresh

Question 4- The students Table

```
SELECT*
FROM exercise3.student_table2.student;

--Question4:
SELECT student_name,
       score,
       CASE
         WHEN score >= 90 THEN 'A'
         WHEN score BETWEEN 80 AND 89 THEN 'B'
         WHEN score BETWEEN 70 AND 79 THEN 'C'
         WHEN score BETWEEN 60 AND 69 THEN 'D'
         WHEN score < 60 THEN 'F'
       END AS grade
FROM exercise3.student_table2.student;
```

Add parameter

Table +

Q

Y

	A ^B _C student_name	1 ² ₃ score	A ^B _C grade
1	Anna	92	A
2	Ben	76	C
3	Cara	59	F
4	David	83	B
5	Ella	68	D

↓

▼

5 rows | 0.404s runtime [See performance \(1\)](#) ✓ Checked for improvements

Refreshed N

Question 5- The deliveries Table

(5,100);

SELECT*

FROM exercise3.deliveries_table.deliveries;

--QUESTIONS:

SELECT delivery_id,

.....delivery_time_minutes,

CASE

....WHEN delivery_time_minutes<= 30 THEN 'Fast'

....WHEN delivery_time_minutes BETWEEN 31 AND 60 THEN 'On time'

....WHEN delivery_time_minutes>60 THEN 'Late'

END AS performance

FROM exercise3.deliveries_table.deliveries;

Add parameter

Table

+

🔍

🔧

📄

🔒

⬆

✕

	123 delivery_id	123 delivery_time_minutes	A8C performance
1	1	45	On time
2	2	80	Late
3	3	30	Fast
4	4	65	Late
5	5	100	Late

⬇

⌵

5 rows

0.599s runtime

[📊 See performance \(1\)](#)

✓ Checked for improvements

Refreshed Now

Question 6- The tickets Table

▶ Run selected (1000) ✓ Just now (<1s) workspace default

```
-- Question 6
(5, 'password reset', 1);

SELECT*
FROM exercise3.tickets_table2.tickets;

--Question 6
SELECT issue_type,
       priority,
CASE
WHEN priority=3 THEN 'High'
WHEN priority=2 THEN 'Medium'
WHEN priority=1 THEN 'Low'
END AS Priority_label
FROM exercise3.tickets_table2.tickets;
```

Add parameter

Table				+	Q	F	I	□	⌂	^	X
	issue_type	priority	Priority_label								
1	login issue	1	Low								
2	server down	3	High								
3	slow system	2	Medium								
4	email error	2	Medium								
5	password reset	1	Low								

↓

5 rows | 0.418s runtime | [See performance \(1\)](#) | ✓ Checked for improvements

Refreshed No

Question 7- The attendance Table

▶ Run all (1000) ✓ Just now (1s) workspace: default

⋮

Edit

(3,30),
(4,65),
(5,100);

--Question 7 calculate each student's attendance and classify the results

SELECT student_id,
 (days_present/total_days)*100 AS attendance_percentage,
CASE
 WHEN attendance_percentage>=90 THEN 'Excellent'
 WHEN attendance_percentage BETWEEN 75 AND 89 THEN 'Good'
 WHEN attendance_percentage <75 THEN 'Need Improvement'
END AS attendance_status
FROM exercise3.classact.attendance;

Add parameter

Table +

🔍

🔧

📄

🔒

⌵

✕

	1.3 student_id	1.2 attendance_percentage	1.1 attendance_status
1	1	90	Excellent
2	2	60	Need Improvement
3	3	96	Excellent
4	4	50	Need Improvement
5	5	100	Excellent

⌵

5 rows | 0.934s runtime [See performance \(1\)](#) ✓ Checked for improvements

Refreshed Now

Question 8-The products_inventory Table

▶ Run all (1000) ✓ Just now (1s) workspace default

Edit

```
(4,10),
(5,3);

SELECT*
FROM exercise3.product_inventory_table.product_inventory;

SELECT product_id,
       stock_qty,
CASE
  WHEN stock_qty= 0 THEN 'Out_of_stock'
  WHEN stock_qty BETWEEN 1 AND 5 THEN 'Low stock'
  WHEN stock_qty> 5 THEN 'In stock'
END AS stock_status
```

Add parameter

Table +

	product_id	stock_qty	stock_status
1	1	5	Low stock
2	2	0	Out_of_stock
3	3	25	In stock
4	4	10	In stock
5	5	3	Low stock

↓

5 rows | 0.632s runtime [See performance \(1\)](#) ✓ Checked for improvements

Refreshed Now

Question 9 -The classes Table

```
SELECT*
FROM exercise3.classes_table.classes;

SELECT subject,
       enrolled_students,
CASE
  WHEN enrolled_students>= 25 THEN 'Large'
  WHEN enrolled_students BETWEEN 10 AND 24 THEN 'Medium'
  WHEN enrolled_students<10 THEN 'Small'
END AS class_size_category
FROM exercise3.classes_table.classes;
```

Add parameter

Table +

Q

Y

⌵

□

⌵

^

×

	A _C subject	I ₃ enrolled_students	A _C class_size_category
1	Math	30	Large
2	English	25	Large
3	Science	15	Medium
4	Art	5	Small
5	History	20	Medium

↓

▼

5 rows

0.361s runtime

[See performance \(1\)](#)

✓ Checked for improvements

Refreshed Now

⌵

Question 10- The payments Table

```
SELECT*
FROM exercise3.payments_table.payments;

SELECT payment_id,
       payment_method,
       amount,
CASE
  WHEN payment_method ='cash' AND amount>=200 THEN 'Eligible for discount'
  ELSE 'Not eligible for discount'
END AS discount_eligibility
FROM exercise3.payments_table.payments;
```

Generate (Ctrl + I)

Add parameter

← Results 2 of 2 →

Table

+

Q Y H □ E ^ X

	1 ² payment_id	A ^B payment_method	.00 amount	A ^B discount_eligibility
1	1	card	50.00	Not eligible for discount
2	2	cash	200.00	Eligible for discount
3	3	card	150.00	Not eligible for discount
4	4	Paypal	75.00	Not eligible for discount
5	5	cash	300.00	Eligible for discount

↓ ▾

5 rows

0.916s runtime

[See performance \(2\)](#)

✓ Checked for improvements

Refreshed Now

